



Dokumentacja Techniczna

RecipeBox

Manager Przepisów Kulinarnych

Projekt wykonany na potrzeby przedmiotu *Aplikacje Mobilne*.

Przygotował: **Julian Machowski**

Klasa: **4TP**

Rok szkolny: 2025/26

1. Opis aplikacji

RecipeBox to mobilna aplikacja na system *Android* służąca do zarządzania przepisami kulinarnymi. Umożliwia ona przeglądanie, dodawanie oraz usuwanie własnych przepisów. Dodatkowo pozwala na przeglądanie receptur z zewnętrznego *API Spoonacular*.

Wybrany temat projektu: **RecipeBox (Manager Przepisów)**

Funkcje kluczowe: Przeliczanie składników na liczbę porcji i generowanie listy zakupów.

Testy: Skalowanie jednostek miar, walidacja składników.

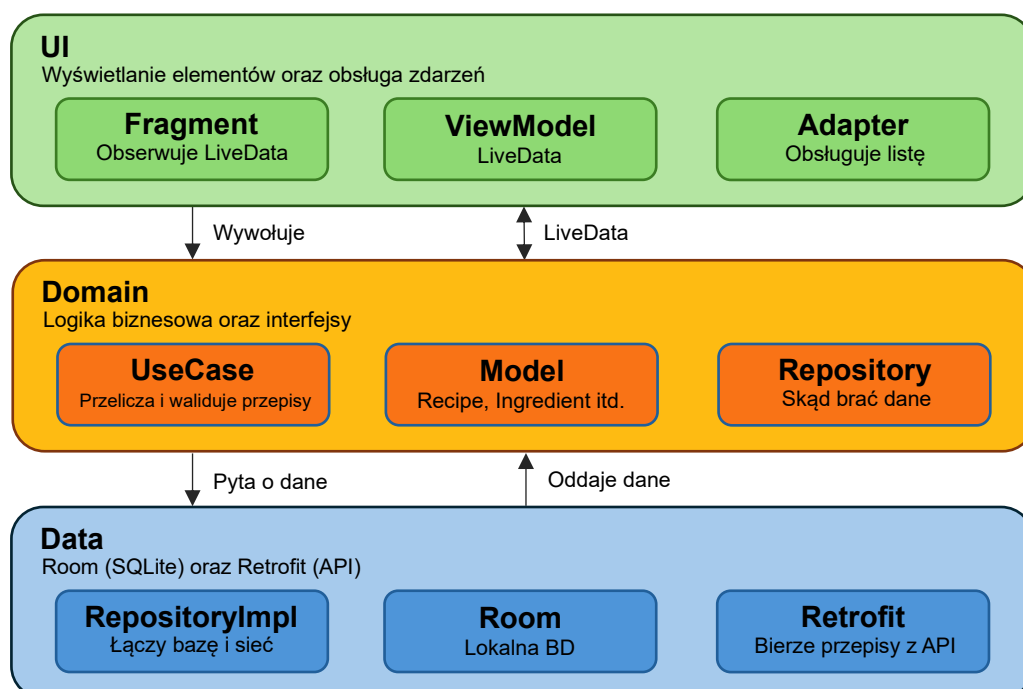
2. Architektura aplikacji

Aplikacji jest w pełni zbudowana w oparciu o wzorzec *MVVM (Model-View-ViewModel)* z zachowaniem zasad *Clean Architecture*.

2.1 Warstwy

Warstwa	Rola	Główne klasy
UI	Wyświetlanie danych i obsługa zdarzeń.	Fragment, ViewModel, Adapter
Domain	Logika biznesowa i interfejsy.	UseCase, Model, Repository
Data	Źródła danych – Room, API (Retrofit)	Dao, Entity, RepositoryImpl, API

2.2 Diagram



3. Użyte technologie

Technologia	Wybrana Technologia
Język	Java 8
Architektura	MVVM + Clean Architecture
Baza danych	Room (SQLite)
Obsługa API	Retrofit 2 + OkHttp3
API	Spoonacular Food API
Testy	JUnit 5.10.1 (Jupiter), JUnit 4 (do Espresso)
Testy (Mock)	Mockito 5.8.0 + Mockito-JUnit-Jupiter
Nawigacja	Jetpack Navigation Component
UI	Material Design 3
Min. Android	API 26 (Android 8.0 Oreo)
Target SDK	API 34 (Android 14)

4. Struktura projektu

```

RecipeBox/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/recipebox/
│   │   │   │   ├── data/                # DATA
│   │   │   │   │   ├── local/            # Baza Room (DAO, Database)
│   │   │   │   │   └── repository/        # Implementacje repozytoriów
│   │   │   │   ├── domain/              # DOMAIN
│   │   │   │   │   ├── model/            # Encje (Recipe, Ingredient)
│   │   │   │   │   └── usecase/          # Przypadki użycia (Scale, Delete)
│   │   │   │   └── ui/                   # UI
│   │   │   │       ├── home/              # Ekran główny
│   │   │   │       ├── myrecipes/         # Moje przepisy
│   │   │   │       ├── addrecipe/         # Dodawanie przepisów
│   │   │   │       ├── detail/            # Szczegóły przepisu
│   │   │   │       ├── shoppinglist/      # Listy zakupów
│   │   │   │       └── MainActivity.java   # Główne okno aplikacji
│   │   └── res/                          # ZASOBY
│   │       ├── layout/                   # Pliki XML widoków
│   │       ├── navigation/               # Graf nawigacji (nav_graph.xml)
│   │       ├── menu/                     # Menu dolne i górne
│   │       └── values/                   # Kolory, napisy, style
│   │           └── AndroidManifest.xml
│   └── test/                             # Testy
└── build.gradle.kts                       # Konfiguracja modułu app
  
```

5. Opis kluczowych klas

5.1 Warstwa UI

RecipeDetail Fragment + RecipeDetailViewModel

Najbardziej rozbudowany ekran. Przyciski +/- wywołują `scaleToServings()` w ViewModelu, dodawanie składników do listy zakupów.

5.2 Warstwa Domain

Recipe.java

Centralna encja domeny reprezentująca przepis. Oprócz przechowywania danych (m.in. listy składników i kroków), zawiera kluczową logikę biznesową odpowiedzialną za niemutowalne (kopia przepisu) skalowanie porcji potrawy.

Ingredient.java

Encja opisująca pojedynczy składnik. Posiada wbudowaną logikę do przeliczania jednostek miary, formatowanie tekstowe ilości produktów itp.

ScaleRecipeUseCase.java

Pełni rolę łącznika między modelami, odpowiadając za transformację przepisu w listę zakupów oraz walidację reguł biznesowych przed zapisem.

5.3 Warstwa Data

AppDatabase.java

Główny punkt dostępu do lokalnej bazy danych SQLite przez bibliotekę Room.

RecipeRepositoryImpl.java

Implementacja *RecipeRepository*, która ukrywa przed resztą aplikacji źródło pochodzenia danych. Koordynuje przepływ informacji między lokalną bazą danych (Room) oraz zewnętrznym serwisem API (Retrofit).

6. Schemat bazy danych (SQLite / Room)

recipes

Kolumna	Typ	Opis
id	INTEGER	PK, AUTOINCREMENT
name	TEXT	Nazwa potrawy
description	TEXT	Opis przepisu
imageUrl	TEXT	Link/Ścieżka do zdjęcia
servings	INTEGER	Liczba porcji
prepTimeMinutes	INTEGER	Czas przygotowania w minutach
cookTimeMinutes	INTEGER	Czas gotowania/pieczczenia w minutach
difficulty	TEXT	Trudność
category	TEXT	Kategoria
isLocal	INTEGER	[0/1] czy przepis lokalny
ingredients	TEXT	Lista składników jako JSON
steps	TEXT	Lista kroków jak JSON

shopping_lists

Kolumna	Typ	Opis
id	INTEGER	PK, AUTOINCREMENT
name	TEXT	Nazwa listy
createdAt	INTEGER	Data utworzenia w ms

shopping_items

Kolumna	Typ	Opis
id	INTEGER	PK, AUTOINCREMENT
listId	INTEGER	FK (shopping_lists.id) jako CASCADE
name	TEXT	Nazwa składnika
amount	TEXT	Ilość
checked	TEXT	[0/1] czy zaznaczony

7. Testy

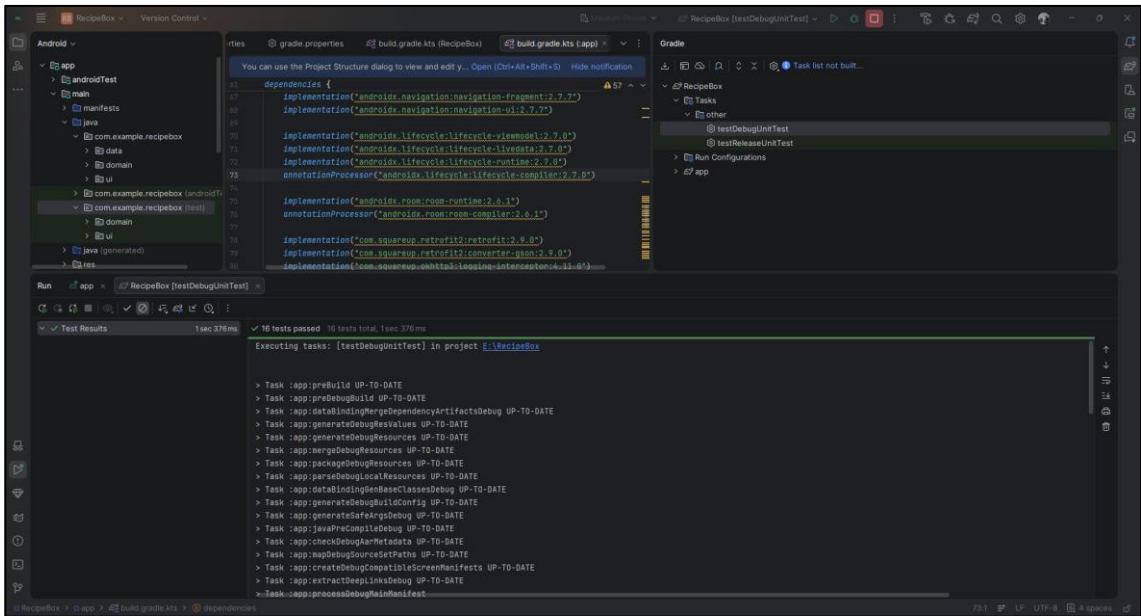
7.1 Lista testów

Lp.	Test	Co sprawdzają asercje?	Asercje	Oczekiwany wynik	Status
ScaleRecipeUseCaseTest					
1.	testValidationFailEmptyName	Rozmiar listy oraz dokładne ilości składników	3	Lista 2 elementowa, wartości: 400.0 i 4.0	✓
2.	testScalingWithInvalidServings	Czy system blokuje 0 porcji (Exception)	1	Rzucenie <i>IllegalArgumentException</i>	✓
3.	testGenerateShoppingList	Nazwę listy, liczbę pozycji i format tekstu	3	"Weekly Shopping", 2 poz., "400 g Pasta"	✓
4.	testValidationSuccess	Czy poprawny przepis przechodzi walidację	1	isValid = true	✓
5.	testValidationFailEmptyName	Czy flaga to false i czy treść błędu jest dobra	2	isValid = false, błąd: "Recipe name is required"	✓
6.	testValidationFailNoIngredients	Czy flaga to false i czy treść błędu jest dobra	2	isValid = false, błąd: "Recipe must contain..."	✓
IngredientTest					

7.	<i>testConversionGramToKg</i>	Poprawność matematyczną i zmianę jednostki	2	Wartość 1.5, jednostka: KILOGRAM	✓
8.	<i>testConversionMIToL</i>	Przeliczenie mililitrów na litry	1	Wartość 0.5 (dla 500 ml)	✓
9.	<i>testSmartUnit</i>	Zmianę na wyższą jednostkę i nową wartość	2	Jednostka: KILOGRAM, wartość: 1.2	✓
10.	<i>testFormat</i>	Formatowanie tekstu dla różnych produktów	2	Napisy: "5 g Salt" oraz "2 pcs Apple"	✓
RecipeTest					
11.	<i>testScaleIngredients</i>	Skalowanie wewnątrz klasy <i>Recipe</i>	2	Zmniejszona ilość składnika (z 200 na 100)	✓
HomeViewModelTest					
12.	<i>load_updatesTotalCount</i>	Czy licznik przepisów w UI jest poprawny	1	Wartość w LiveData równa 2	✓
13.	<i>load_fetchesRecipeFromApi</i>	Wywołanie API i nazwę pobranego dania	2	Wywołanie <i>searchRecipes</i> , nazwa: "Test API Recipe"	✓
ApiRecipesViewModelTest					
14.	<i>search_updatesSuccess</i>	Dane w LiveData oraz wyłączenie ładowania	3	Lista niepusta, loading = false	✓
15.	<i>search_updatesError</i>	Komunikat o błędzie oraz wyłączenie ładowania	2	Komunikat: "Network Error", loading = false	✓
16.	<i>loadRecipe_updatesData</i>	Nazwę przepisu i liczbę porcji w szczegółach	2	Nazwa: "Test Recipe", porcje: 4	✓

Aplikacja dodatkowo posiada automatyczne testy UI wykonane przy użyciu biblioteki *Espresso*. Symulują one interakcję użytkownika z interfejsem m.in. poprawność nawigacji czy dodawanie nowych przepisów.

7.2 Wyniki testów (Android Studio)



7.3 Wyniki testów (Raport HTML)

Test Summary

16
tests

0
failures

0
ignored

1.376s
duration

100%
successful

Packages

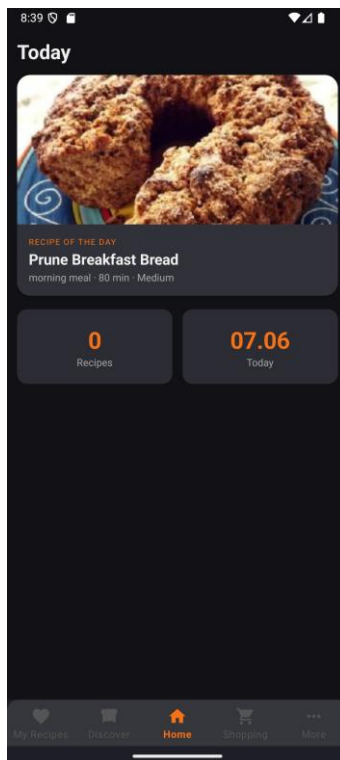
Classes

Class	Tests	Failures	Ignored	Duration	Success rate
com.example.recipebox.domain.model.IngredientTest	4	0	0	0.005s	100%
com.example.recipebox.domain.model.RecipeTest	1	0	0	0s	100%
com.example.recipebox.domain.usecase.ScaleRecipeUseCaseTest	6	0	0	0.003s	100%
com.example.recipebox.ui.apirecipes.ApiRecipesViewModelTest	2	0	0	0.991s	100%
com.example.recipebox.ui.detail.RecipeDetailViewModelTest	1	0	0	0.148s	100%
com.example.recipebox.ui.home.HomeViewModelTest	2	0	0	0.229s	100%

Generated by Gradle 9.1.0 at 7 cze 2026, 22:35:37

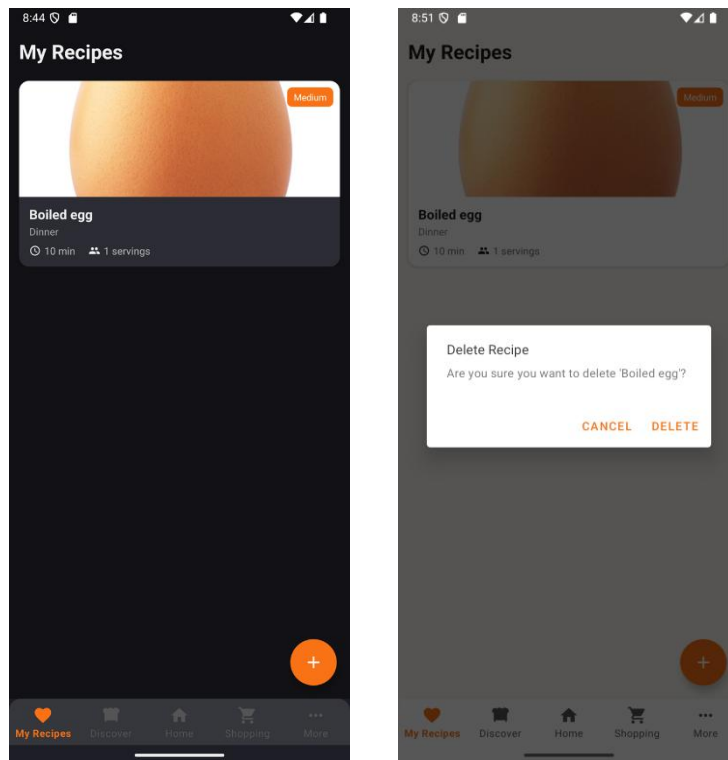
7 Widoki aplikacji

7.1 Ekran główny



Główny ekran aplikacji zawierający pole z losowym przepisem, licznikiem własnych przepisów oraz dzisiejszą datą.

7.2 Moje przepisy i usuwanie przepisu.

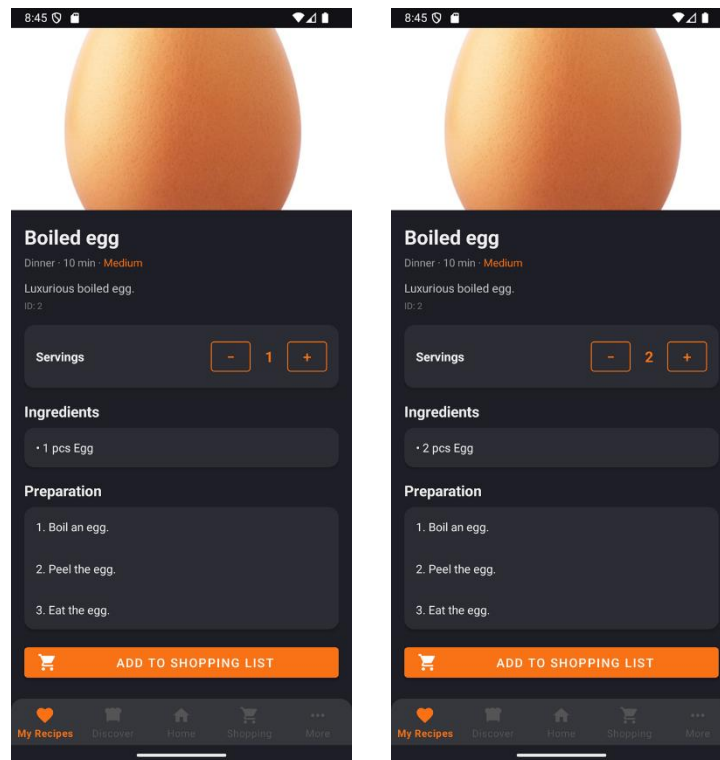


Ekran listę przepisów użytkownika wraz z przyciskiem „Dodaj nowy przepis”. Gdy przytrzyma się kafelek przepisu można po potwierdzeniu go usunąć (PRAWO).

7.3 Dodawanie nowego przepisu

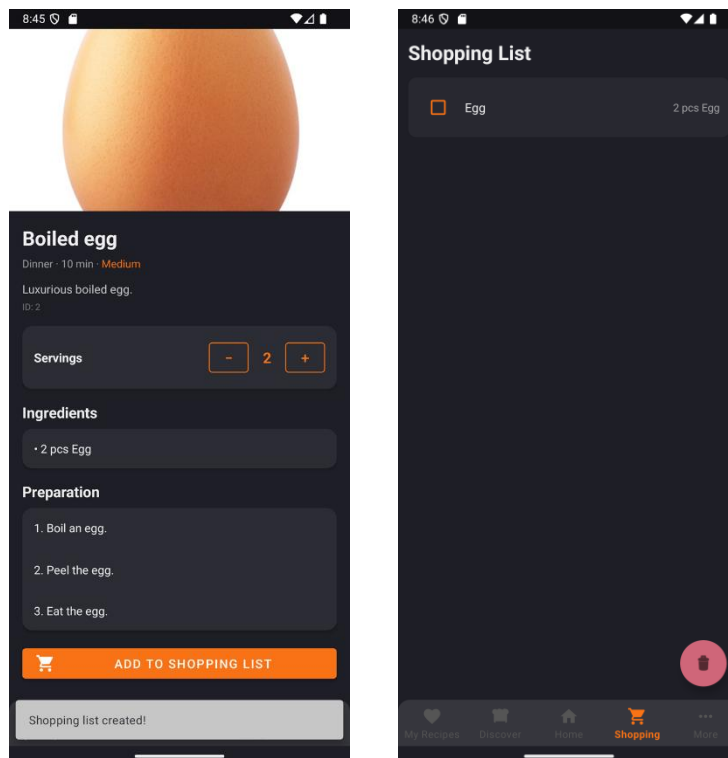
Ekran dodawania przepisu przez użytkownika.

7.4 Szczegóły przepisu i skalowanie porcji



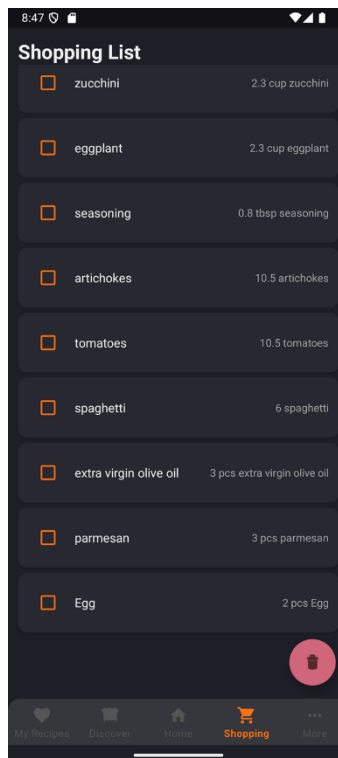
Ekran szczegółów przepisu wraz z funkcją skalowania porcji. Klikamy +/- i ilość składników automatycznie się zmienia (ekran z lewej – PRZED, po prawej – PO).

7.5 Lista zakupów oraz dodawanie do niej produktów



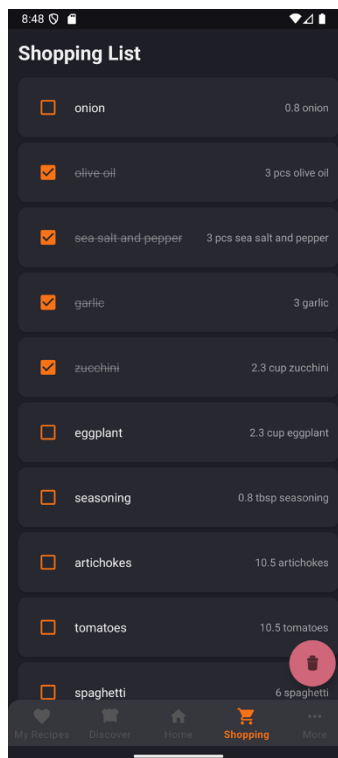
Ekran listy zakupów (po prawej) po kliknięciu przycisku „Add to shopping list”.

7.6 Grupowanie produktów w liście zakupowej



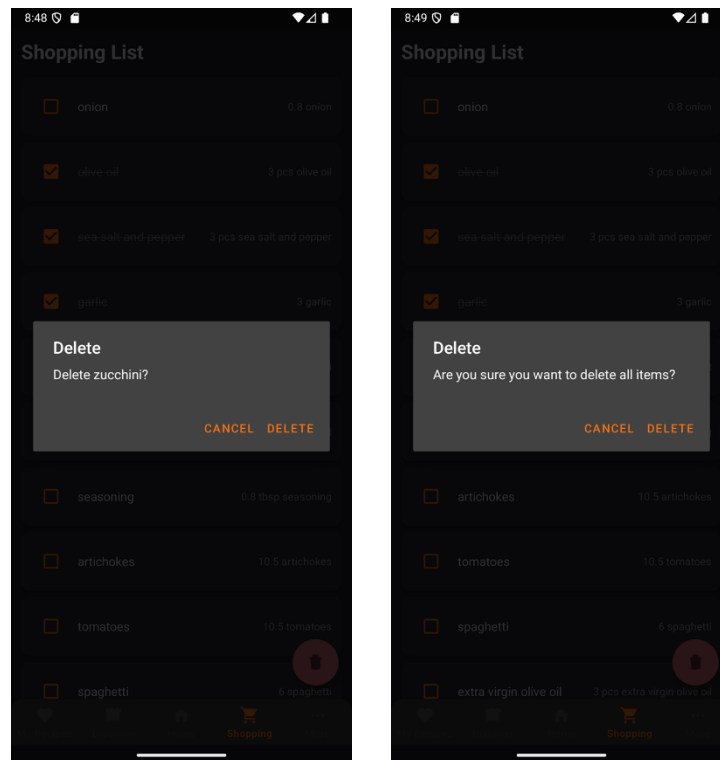
Po dodaniu produktów z innego przepisu produkty należące do niego pojawiają się u góry, jeden pod drugim (nie mieszają się z produktami innych przepisów).

7.7 Zaznaczanie zakupionych/posiadanych produktów



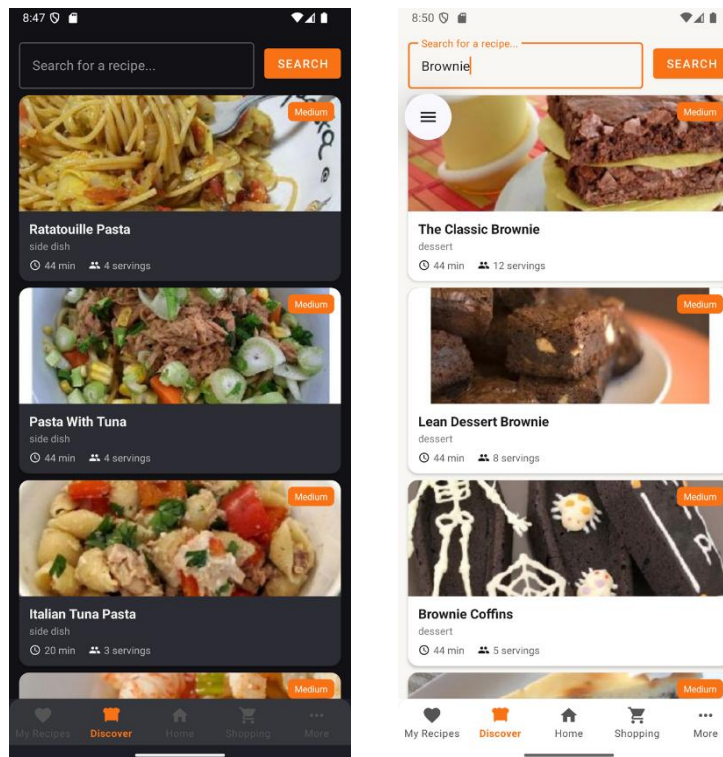
Istnieje możliwość zaznaczenia (NIE USUNIĘCIA) produktu zakupionego/posiadanego (przekreślenie).

7.8 Usunięcie przedmiotów z listy zakupów



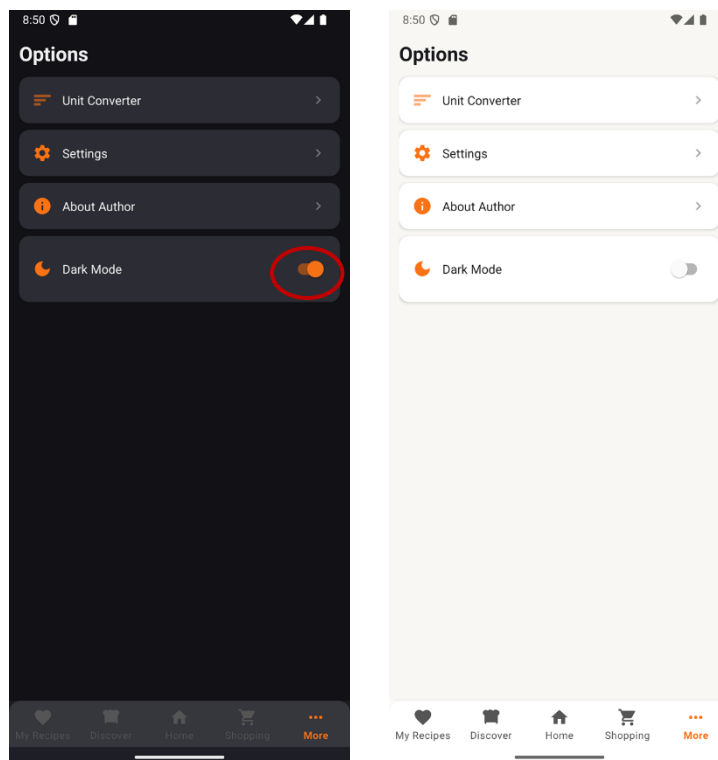
Istnieją również dwa sposoby na usunięcie produktu z listy: 1. Przytrzymujemy pole produktu i potwierdzamy usunięcie. (LEWO) 2. Klikamy przycisk „Usuń” w prawym dolnym rogu i usuwamy całą listę zakupową. (PRAWO)

7.9 Przepisy z API oraz funkcja wyszukiwania przepisów



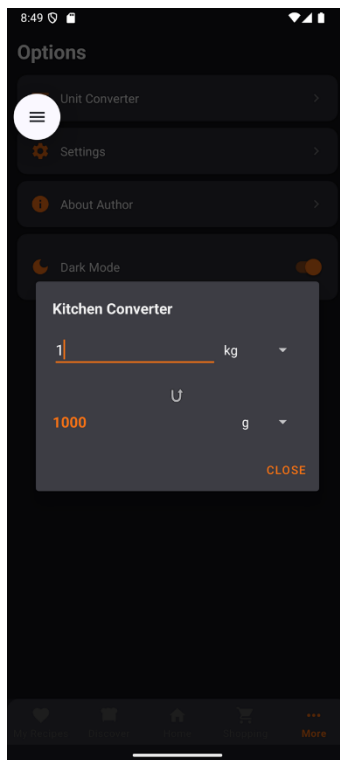
Ekran przepisów pobieranych z API. Po PRAWIEJ można zobaczyć działanie wyszukiwarki przepisów.

7.10 Ekran więcej oraz zmiana motywu



Ekran „więcej” z kafelkami: „Konwerter”, „Ustawienia”- obecnie nic, „O Autorze”, „Zmiana motywu” – automatycznie ciemny (z możliwością zmiany na jasny i odwrotnie – efekt po PRAWEJ)

7.11 Konwerter jednostek



Zamienia jednostki według grup (objętość, masa itp.).

8 Uruchomienie

8.1 Wymagania

- Android Studio: Hedgehog (2023.1.1) lub nowszy
- JDK 1.8 (Java 8)
- Android SDK API 26 i więcej
- Klucz Spoonacular API (50 requestów na dzień)

8.2 Uruchomienie i konfiguracja

- Otwórz projekt w Android Studio
- W pliku *local.properties* podmień KLUCZ_API na własny (dostarczę w mailu dwa)
- File Sync Project with Gradle Files
- Uruchom na emulatorze lub urządzeniu (API 26+)

9 Podsumowanie

Projekt był czasochłonny i bardzo rozbudowany, dokładnie tak jak zakładałem na początku. Praca przebiegała różnie – raz szło lepiej a raz gorzej, ale ogólnie robiło się go całkiem sprawnie. Efekt końcowy uważam za zadowalający (aplikacja zawiera wszystkie założone wymagania). Liczę, że zasłużyłem na ocenę celującą. 😊